

Experiment No. 6

Aim: Implementation of fuzzy set Properties.

Theory:

Fuzzy operations are fundamental in fuzzy logic systems and are used to manipulate fuzzy sets. These operations include union, intersection, and complement, which correspond to logical OR, AND, and NOT operations, respectively, in classical set theory. However, in fuzzy logic, the operations are defined with degrees of membership rather than binary (0 or 1) values.

Here's an implementation of the basic fuzzy operations in Python, along with examples of how to apply them:

1. Union (Fuzzy OR)

The degree of membership in the union of two fuzzy sets $\tilde{A}$ and $\tilde{B}$ is given by the maximum of the degrees of membership of the elements in each set:

$$\mu_{\tilde{A} \cup \tilde{B}}(y) = \mu_{\tilde{A}} \vee \mu_{\tilde{B}} \quad \forall y \in U$$

2. Intersection (Fuzzy AND)

The degree of membership in the intersection of two fuzzy sets $\tilde{A}$ and $\tilde{B}$ is given by the minimum of the degrees of membership of the elements in each set:

$$\mu_{\tilde{A} \cap \tilde{B}}(y) = \mu_{\tilde{A}} \wedge \mu_{\tilde{B}} \quad \forall y \in U$$

3. Complement (Fuzzy NOT)

The degree of membership in the complement of a fuzzy set $\tilde{A}$ is given by:

$$\mu_{\tilde{A}} = 1 - \mu_{\tilde{A}}(y) \quad y \in U$$

Properties of Fuzzy Sets:

Let us discuss the different properties of fuzzy sets.

1. Commutative Property

Having two fuzzy sets $\tilde{A}$ and $\tilde{B}$, this property states -

$$\tilde{A} \cup \tilde{B} = \tilde{B} \cup \tilde{A}$$

$$\tilde{A} \cap \tilde{B} = \tilde{B} \cap \tilde{A}$$

2. Associative Property for Intersection ($\cap$):

For any three fuzzy sets A, B, and C, the associative property for intersection is:

$$(A \cap B) \cap C = A \cap (B \cap C)$$

- Here, $\cap$ represents the intersection operation, which is typically implemented using a t-norm (like the minimum or product).

Associative Property for Union ($\cup$):

For any three fuzzy sets A, B, and C, the associative property for union is:

$$(A \cup B) \cup C = A \cup (B \cup C)$$

- Here, $\cup$ represents the union operation, which is typically implemented using a t-conorm (like the maximum or probabilistic sum).

3. Distributive Property

Having three fuzzy sets $\sim A$, $\sim B$ and $\sim C$, this property states –

$$\tilde{A} \cup (\tilde{B} \cap \tilde{C}) = (\tilde{A} \cup \tilde{B}) \cap (\tilde{A} \cup \tilde{C})$$

$$\tilde{A} \cap (\tilde{B} \cup \tilde{C}) = (\tilde{A} \cap \tilde{B}) \cup (\tilde{A} \cap \tilde{C})$$

4. Idempotency Property

For any fuzzy sets $\sim A$, this property states –

$$\tilde{A} \cup \tilde{A} = \tilde{A}$$

$$\tilde{A} \cap \tilde{A} = \tilde{A}$$

Input:

Below is a Python implementation of these basic fuzzy operations:

Step 1: Importing Necessary Packages

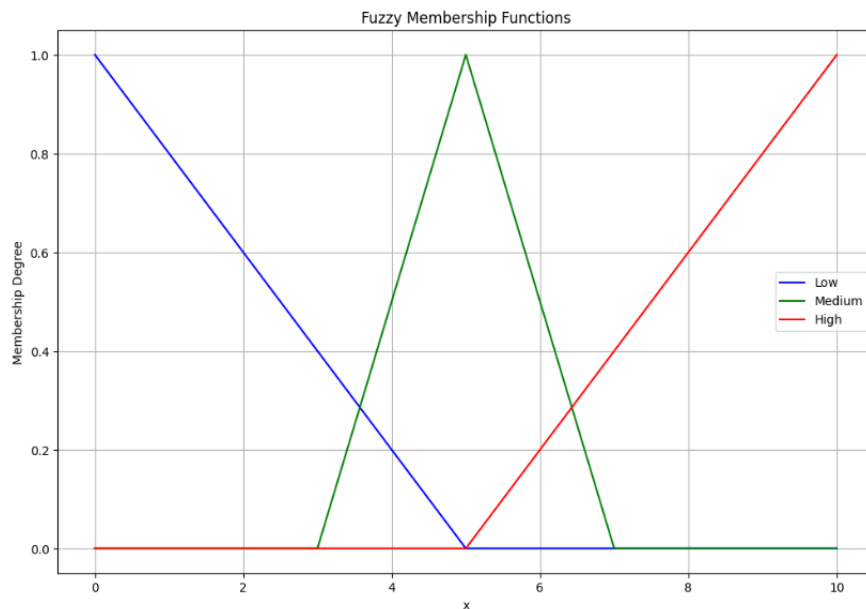
```
import numpy as np
import matplotlib.pyplot as plt
import skfuzzy as fuzz
from skfuzzy import control as ctrl
```

Step 2: Define Universe of Discourse for Fuzzy Operation

```
[2]: x = np.arange(0, 11, 1)

# Define fuzzy variables with triangular membership functions
A = fuzz.trimf(x, [0, 0, 5]) # Fuzzy set A: Low
B = fuzz.trimf(x, [3, 5, 7]) # Fuzzy set B: Medium
C = fuzz.trimf(x, [5, 10, 10]) # Fuzzy set C: High

# Plot membership functions
plt.figure(figsize=(12, 8))
plt.plot(x, A, label='Low', color='blue')
plt.plot(x, B, label='Medium', color='green')
plt.plot(x, C, label='High', color='red')
plt.title('Fuzzy Membership Functions')
plt.xlabel('x')
plt.ylabel('Membership Degree')
plt.legend()
plt.grid()
plt.show()
```

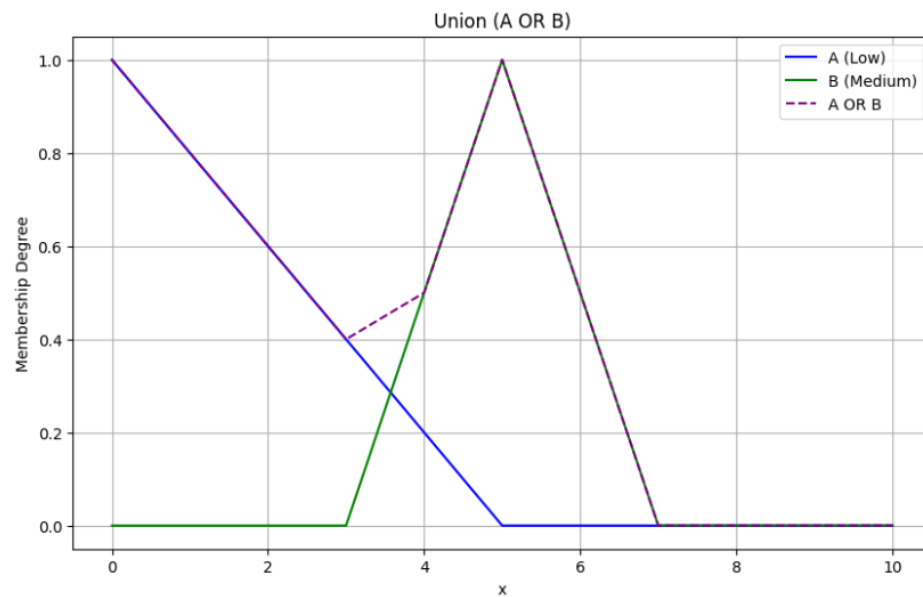


Step 3: Fuzzy Operation

1. Union of A and B

```
[3]: A_or_B = np.maximum(A, B)

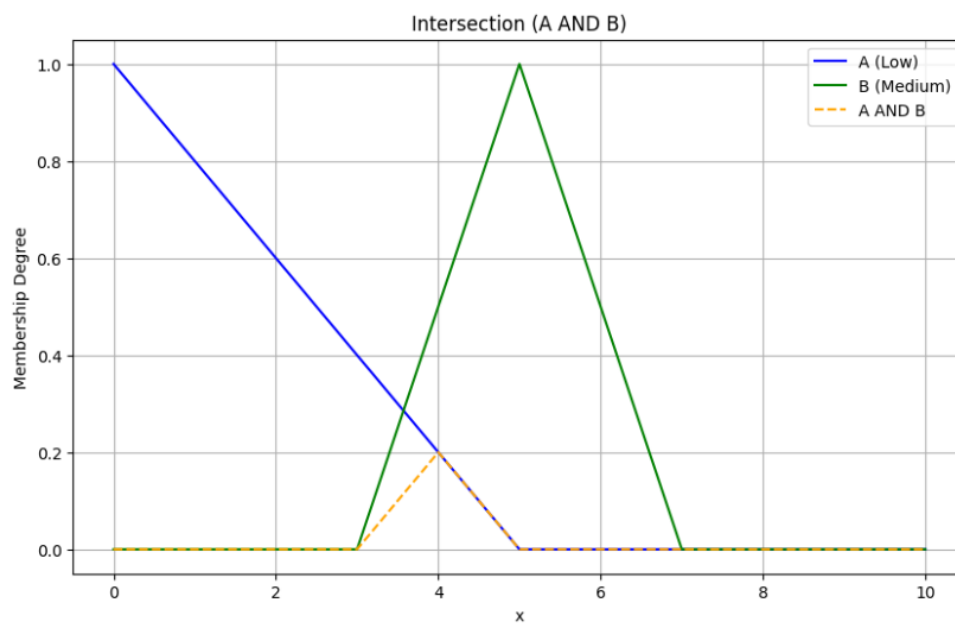
plt.figure(figsize=(10, 6))
plt.plot(x, A, label='A (Low)', color='blue')
plt.plot(x, B, label='B (Medium)', color='green')
plt.plot(x, A_or_B, label='A OR B', color='purple', linestyle='--')
plt.title('Union (A OR B)')
plt.xlabel('x')
plt.ylabel('Membership Degree')
plt.legend()
plt.grid()
plt.show()
```



2. Intersection of A and B

```
[4]: A_and_B = np.minimum(A, B)

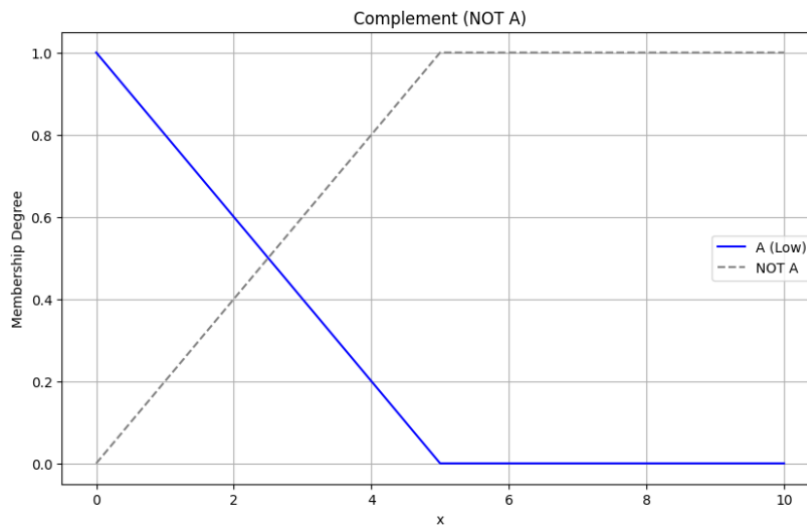
plt.figure(figsize=(10, 6))
plt.plot(x, A, label='A (Low)', color='blue')
plt.plot(x, B, label='B (Medium)', color='green')
plt.plot(x, A_and_B, label='A AND B', color='orange', linestyle='--')
plt.title('Intersection (A AND B)')
plt.xlabel('x')
plt.ylabel('Membership Degree')
plt.legend()
plt.grid()
plt.show()
```



3. Complement of A

```
[5]: A_not = 1 - A

plt.figure(figsize=(10, 6))
plt.plot(x, A, label='A (Low)', color='blue')
plt.plot(x, A_not, label='NOT A', color='gray', linestyle='--')
plt.title('Complement (NOT A)')
plt.xlabel('x')
plt.ylabel('Membership Degree')
plt.legend()
plt.grid()
plt.show()
```



Step 4: Fuzzy Set Properties

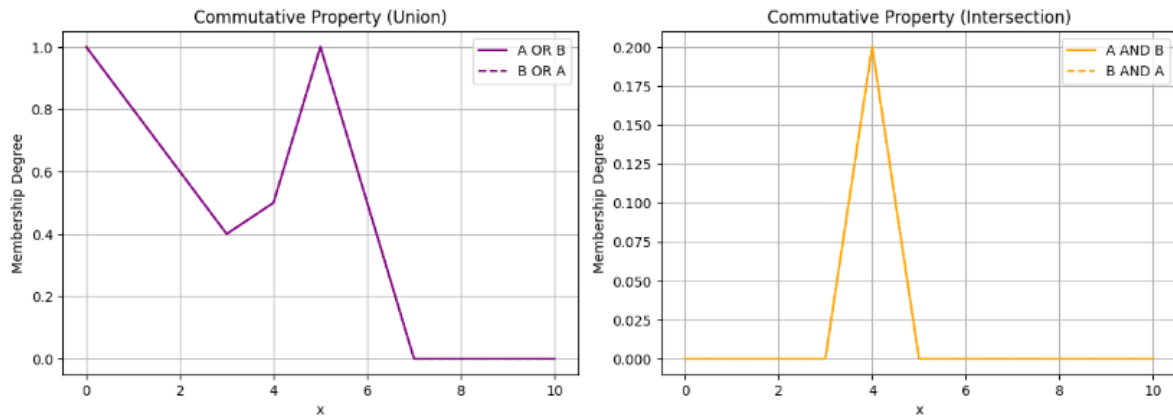
1. Commutative Property

```
[7]: A_or_B_commutative = np.maximum(B, A)
A_and_B_commutative = np.minimum(B, A)
plt.figure(figsize=(12, 8))

plt.subplot(2, 2, 1)
plt.plot(x, A_or_B, label='A OR B', color='purple')
plt.plot(x, A_or_B_commutative, label='B OR A', linestyle='--', color='purple')
plt.title('Commutative Property (Union)')
plt.xlabel('x')
plt.ylabel('Membership Degree')
plt.legend()
plt.grid()

plt.subplot(2, 2, 2)
plt.plot(x, A_and_B, label='A AND B', color='orange')
plt.plot(x, A_and_B_commutative, label='B AND A', linestyle='--', color='orange')
plt.title('Commutative Property (Intersection)')
plt.xlabel('x')
plt.ylabel('Membership Degree')
plt.legend()
plt.grid()

plt.tight_layout()
plt.show()
```

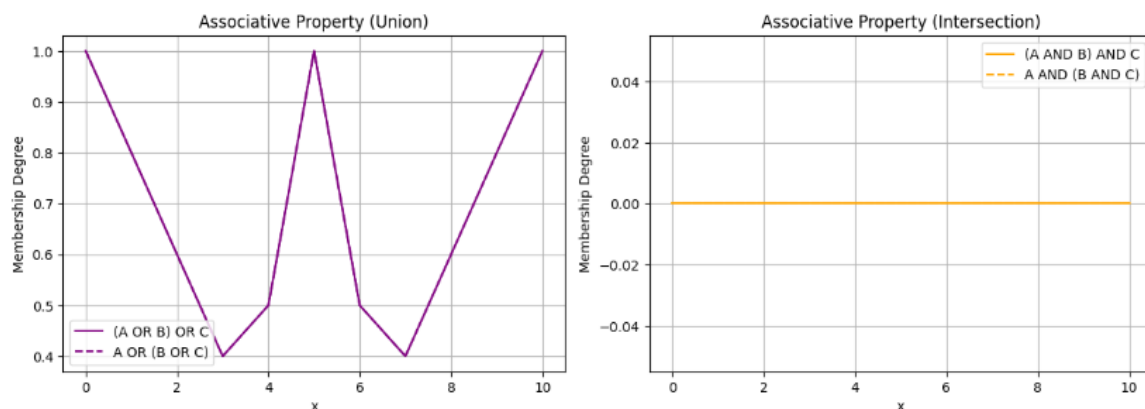


2. Associative Property

```
[8]: A_or_B_or_C = np.maximum(A_or_B, C)
A_or_B_or_C_assoc = np.maximum(A, np.maximum(B, C))
A_and_B_and_C = np.minimum(A_and_B, C)
A_and_B_and_C_assoc = np.minimum(A, np.minimum(B, C))
plt.figure(figsize=(12, 8))

plt.subplot(2, 2, 1)
plt.plot(x, A_or_B_or_C, label='(A OR B) OR C', color='purple')
plt.plot(x, A_or_B_or_C_assoc, label='A OR (B OR C)', linestyle='--', color='purple')
plt.title('Associative Property (Union)')
plt.xlabel('x')
plt.ylabel('Membership Degree')
plt.legend()
plt.grid()

plt.subplot(2, 2, 2)
plt.plot(x, A_and_B_and_C, label='(A AND B) AND C', color='orange')
plt.plot(x, A_and_B_and_C_assoc, label='A AND (B AND C)', linestyle='--', color='orange')
plt.title('Associative Property (Intersection)')
plt.xlabel('x')
plt.ylabel('Membership Degree')
plt.legend()
plt.grid()
plt.tight_layout()
plt.show()
```



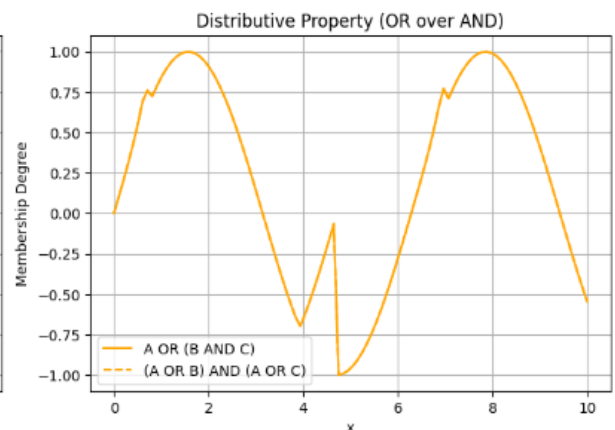
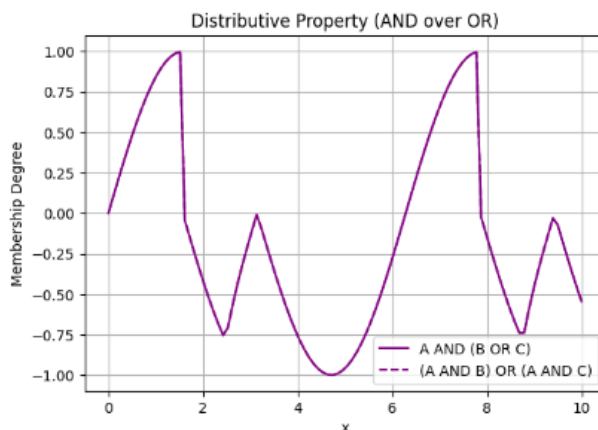
3. Distributive Property

```
[9]: x = np.linspace(0, 10, 100)
A = np.sin(x)
B = np.cos(x)
C = np.tan(x)

A_and_B_or_C = np.minimum(A, np.maximum(B, C))
A_and_B_or_C_combined = np.maximum(np.minimum(A, B), np.minimum(A, C))
A_or_B_and_C = np.maximum(A, np.minimum(B, C))
A_or_B_and_C_combined = np.minimum(np.maximum(A, B), np.maximum(A, C))
plt.figure(figsize=(12, 8))

plt.subplot(2, 2, 1)
plt.plot(x, A_and_B_or_C, label='A AND (B OR C)', color='purple')
plt.plot(x, A_and_B_or_C_combined, label='(A AND B) OR (A AND C)', linestyle='--', color='purple')
plt.title('Distributive Property (AND over OR)')
plt.xlabel('x')
plt.ylabel('Membership Degree')
plt.legend()
plt.grid()

plt.subplot(2, 2, 2)
plt.plot(x, A_or_B_and_C, label='A OR (B AND C)', color='orange')
plt.plot(x, A_or_B_and_C_combined, label='(A OR B) AND (A OR C)', linestyle='--', color='orange')
plt.title('Distributive Property (OR over AND)')
plt.xlabel('x')
plt.ylabel('Membership Degree')
plt.legend()
plt.grid()
plt.tight_layout()
plt.show()
```



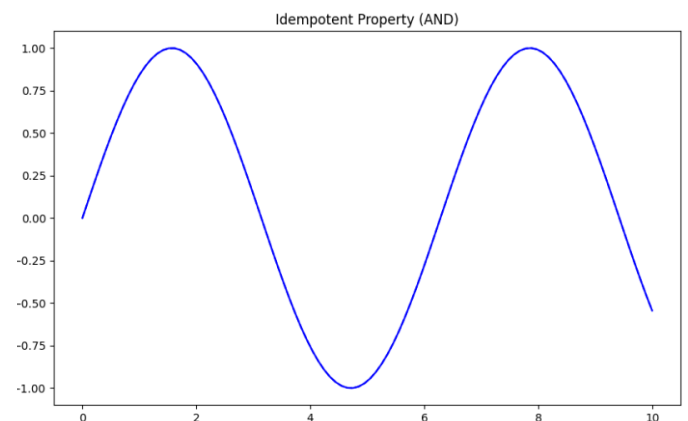
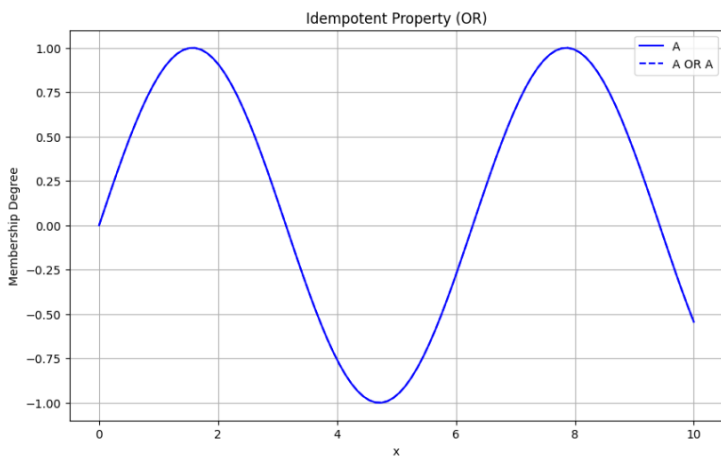
4. Idempotent Property

```
[10]: A_idempotent_or = np.maximum(A, A)
A_idempotent_and = np.minimum(A, A)
plt.figure(figsize=(10, 6))

plt.plot(x, A, label='A', color='blue')
plt.plot(x, A_idempotent_or, label='A OR A', linestyle='--', color='blue')
plt.title('Idempotent Property (OR)')
plt.xlabel('x')
plt.ylabel('Membership Degree')
plt.legend()
plt.grid()

plt.figure(figsize=(10, 6))
plt.plot(x, A, label='A', color='blue')
plt.plot(x, A_idempotent_and, label='A AND A', linestyle='--', color='blue')
plt.title('Idempotent Property (AND)')
```

```
[10]: Text(0.5, 1.0, 'Idempotent Property (AND)')
```



Conclusion: Thus, we have successfully studied and implemented the Fuzzy Set Properties using Example.